Enhanced Code Analysis and Feedback System

Mr. K. Lokeshnath[1][a], N. Harshavardhan Reddy[2][b], M. Naga Sai Navya[3][c], C. Chandrakala[4][d]
[1]*Assistant Professor, Srinivasa ramanujan institute of technology*
[2,3,4] *Student, Srinivasa Ramanujan Institute of Technology*
Department of Computer Science and Engineering
Anantapur, Andhra Pradesh, 515001, India
lokeshnathever@gmail.com,224g1a0526@srit.ac.in, 224g1a0555@srit.ac.in, 234g5a0501@srit.ac.in

KEYWORDS: Enhanced code Analysis, Error Detection, Code Optimization, Code Suggestions and NLTK.

ABSTRACT: Enhanced Code Analysis And Feedback System is a code analysis and improvement tool in the form of a web-based program. This program can be accessed by creating an account through a text input interface, which can be used by all programming languages. The system helps users by submitting their code, which gets sent to an AI system that analyzes the code through the Gemini AI API after the submission process ends. The primary role of the Enhanced Code Analysis And Feedback System program is the detection of various programming problems, which include syntax errors, logical errors, variables, and imports. The program detects errors while it delivers feedback in the form of recommendations to help users improve their code. The system offers its users recommendations towards optimizing their code. The recommendations are offered after all the errors have been rectified. There are three areas in which the recommendations are offered, which include code readability, code performance, and code maintainability. The Enhanced Code Analysis And Feedback System can be considered an all-inclusive code improvement tool. It helps in the improvement of coding dimensions, productivity, and creating better code.

1 INTRODUCTION

The Enhanced Code Analysis And Feedback System is a sophisticated online-based application that uses artificial intelligence in order to support developers in code review activities by detecting bugs and offering optimization solutions. The software development process requires developers to obtain both exact and speedy solutions which should help with code review activities. Human reviewers need to spend multiple hours for their work, but their results become unreliable because human errors cause assessment faults. Developers overlook minor issues that involve logical errors and uninitialized variables, performance constraints that result in product breakdowns, and security threats and operational slowdowns.

The project intends to solve the existing problems by designing an automated system that would handle the review work. The web application will offer users an easy-to-operate text area where they can submit their code, written in multiple programming languages. In the back end, the AI engine will process the code submitted by users with advanced machine learning techniques, including Gemini AI API, among others. The system carries out a comprehensive code assessment procedure which identifies common coding errors such as syntax errors, logical errors, and missing imports and undefined variables while providing developers with detailed feedback that includes correction suggestions.

The system can, therefore, attain optimal performance through its validation test, where no errors are to be detected, as it generates optimization recommendations that will increase comprehension of the codes while at the same time promoting efficiency and viability of the codes. The coding assistant, as it can be referred to, will offer developers coding work as it shows them how they can improve their coding performance through the use of best practices, as indicated in the recommendations. It can offer developers many benefits with its ability to decrease manual reviews, offer reliable error identification, and increase the rate of developing quality codes through Enhanced Code Analysis And Feedback System, as it can evolve with the development of AI and ML, as it can incorporate more forms of programming languages.

[a] https://orcid.org/0009-0009-5288-3144

2 LITERATURE SURVEY

The research on the code review efficiency of LLMs involves an agent trained using entire code repositories, which have information on code review as well as bug tracking. The model identifies code smell, bugs, and style while it makes recommendations for system amendments, which show its use in automatic code quality assessment through LLMs.

The research examines Qodo Copilot and Coderabbit, which are LLM-based tools in code review that impact the processes involved in software development. From the study findings, it was understood that the coding team could clear the automated comments by a rate of about 73.8, which helped in bug fixing as well as in comprehending the quality of the code, though it also demanded extra time in completing the pull requests as it provided false suggestions.

The researchers investigate LLMs code review performance by training agents that use entire code repositories containing both review and bug tracking information. The model detects code smell and bugs, style violations, while giving recommendations for system improvements and performance enhancements, showing how LLMs function as a tool for automatic code quality. A researcher performed an empirical study that examined 22000 automated review comments from GitHub repositories that aimed to establish whether AI-generated review suggestions actually led to code changes. The study yielded different results through its inquiry into comment styles and review settings and development methodologies. The results of the study show that different comment styles and review settings and development methodologies yield different results . The development teams must carry out proper tool implementation for their tools because this is the major requirement that will ensure proper tool integration with their development flows.

The study relies on deep learning techniques to incorporate the semantic code classification of the code review comments using both CodeBERT and LSTM. The system facilitates automation in code reviews, enabling the distinction between such reviews and false positive comments.

The research reviews the current AI code review systems based on the strengths of the system and the limitations of the system including ChatGPT and Gemini API as generative models of a code review system. The paper shows that the AI code review system has two advantages that result in quicker code reviews and higher consistency in the code reviews. However, the paper shows that the system has two weaknesses; the system cannot analyze multiple programming languages and the system's security evaluation has limitations

The investigation examines the opinion on the part of developers based on systems used for the assessment of code quality. The findings show that it allows the majority of users to “attain greater efficiency and productivity improvements.” Still, the users face challenges in trusting how the system works in combination with the workings of systems.

The literature review reveals great developments in AI-driven code review tools that employ machine learning models, including Large Language Models and deep neural networks. The systems show the potential of identifying syntax errors and logical errors while improving code quality through their assessment capabilities. AI tools make development more productive as they ensure code standards through automated functionalities; however, the tools face challenges in working with various programming languages, user trust issues, and difficulties in explaining system operations. Future research should focus on combinations with static analysis techniques to develop reliable and comprehensive code assessment methods for advancing machine automation in software development efficiently.

3 EXISTING SYSTEM

Under the prevailing software development conditions, the code review process mainly relies on the code review through manual peer reviews with the help of basic automation tools. The developers use the version control systems as the tools for submitting code changes. Then the code review process occurs by reviewing the code in such a way that the presence of code bugs and standards can be tracked.

For manual reviewing purposes, static analysis tools like linters and rule sets are often employed. They are very effective at finding syntax errors, cosmetic errors, and defined rule violations. Unfortunately, their ability to analyze code is only surface-level and does not take into account intent, logic

correctness, or design efficiency. As a result, many errors and issues still go unchecked.
Other limitations include a lack of explainable feedback. Most of the automated tools generate warnings without explaining why a problem occurred, let alone an actionable suggestion. This makes it pretty difficult for developers, especially novice ones, to understand and resolve issues accordingly. Also, due to project size and higher commit frequency, manual reviews are not scalable, which leads to late feedback and decreases development productivity.

4 PROPOSED SYSTEM

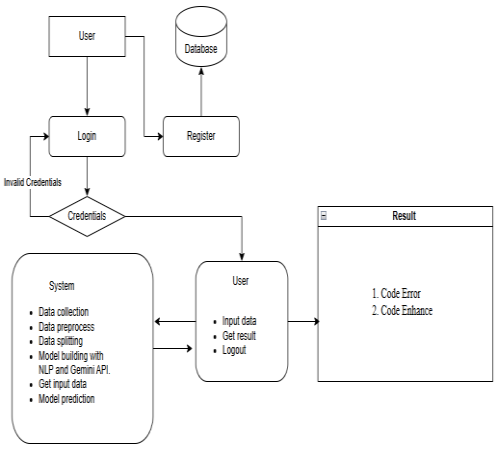
The proposed Enhanced Code Analysis And Feedback System introduces an intelligent and scalable approach to code review by combining static analysis tools with large language models. The system is designed to automatically analyze source code submissions and provide accurate, context-aware, and explainable feedback to developers.
The system consists of three main components: a static analysis module, an AI-based reasoning module, and a feedback generation module. The static analysis module uses established tools to detect syntax errors, code smells, security vulnerabilities, and rule violations. These findings serve as reliable and deterministic inputs for further analysis.
The AI-based reasoning module leverages a large language model to understand the code context, interpret static analysis results, and identify logical issues or improvement opportunities that rule-based tools may miss. It generates human-readable explanations and actionable suggestions, helping developers understand both the issue and its solution.
The feedback generation module is where the results of integrations from the two modules are presented using an interactive interface and/or integrated with existing platform tools during the code review of the pull requests. These tools are language-independent to ensure consistent quality in reviewing any code.
The proposed system, with a focus on automation, will reduce the burden and enhance the quality of defects by improving feedback quality, thereby providing more conducive conditions for coding using this hybrid method, which is explained and robust enough for real-world environments.
The system analyzes the source codes that are entered. These codes are then checked against coding standards and guidelines through an analysis process that remains unbiased and independent, identifying syntactical, logical, and maintainability issues, as well as security risks.
For the review of code to be effective, an intelligent component is required for the purpose of reasoning. This component can interpret the context of the code and produce a user-readable review. Instead of merely identifying problems in the code, reviews can be generated to explain what actually causes the problems. This ensures the user is learning and making improvements in their own coding style. The new system is built with the ability to be integrated easily with modern software development processes. This means it can be integrated with version control systems for code submission and review. This ensures the efficiency of the usage of the new system in the development of software.

5 SYSTEM ARCHITECTURE

The Enhanced Code Analysis And Feedback System architecture follows a modular and scalable pattern by effectively integrating static code analysis tools with artificial intelligence. The architecture of this system follows a layered pattern. The major components of this system include the user interface layer, analysis layer, artificial intelligence layer, and data management layer.
The user interface layer provides interaction with the developers through a web interface or via version control systems like GitHub or GitLab. The developers commit their code or make pull requests, which are then automatically forwarded to the analysis engine. This layer shows the developers' review comments, suggestions, and explanations.
At the analysis level, there are static code analysis tools, which handle the inspection of the source code by searching for syntax errors, code smells, security vulnerabilities, and any deviations from programming standards. At this level, reports are produced, which are reliable input data for subsequent analysis. The deterministic property of code inspection ensures precision and consistency in identifying precisely defined code anomalies.

The reasoning layer makes use of a high-level language model for understanding the static analysis

tool results and for comprehending the contextual intent behind the code written by the developer. The layer also detects logical, design, and improvement flaws that are not identified by rule-based tools.
The data management layer houses code review reports, review history, and configuration rules. Such details are used for tracking improvements, identifying trends, and refining reviews. The proposed architecture allows for effective and intelligent code analysis while providing extensibility to practical software development scenarios.



6 METHODS AND MATERIALS

The system has mechanisms for handling the submissions of the codes through an automatic processing system, and the processing is done using artificial intelligence models. The backend of the system has the following components, which would be explained in detail.

6.1 User Authentication and Authorization

User Login/Registration: The application makes use of a database verification feature for user verification while trying to access the system through the login and registration processes. The application creates a secure authentication system using Flask security features while providing access to users by completing a login authentication process for users.

Database Interaction: The system maintains a user database containing user submission history and feedback. The interaction with the database takes place when users try to log in or access their previous submitted content as follows.

6.2 Code Submission

Code Parsing: The system parses the code that users submit to the system with the purpose of determining the language that users are using based on that code. Text Processing: The provided code is subject to normalizing processes that eliminate superfluous white spaces and symbols to clean the code for further analysis by AI.

6.3 Code Analysis

The AI system analyzes the codes based on its own AI system that connects its backend processes with the Gemini AI API. The system has numerous pre-trained machine learning models that the Gemini AI API offers to analyze different codes written in different programming languages. The models analyze the codes based on different common programming errors, which include the following:

- The programming language has its own syntax rules that are violated by the user.
- The code is constructed with two types of errors that affect the output of the program:
- The system recognizes both undefined variables and missing library imports since it recognizes all variable and import issues.
- Static analysis tools offer advanced feedback due to their ability to identify extra defects that impact code style, complexity, and maintainability.

6.4 Error Detection and Feedback Generation

Feedback Creation: The AI system begins with the process of detecting errors by using data that allows the system to categorize errors of this particular type into three distinctive categories defined as syntax errors, logical errors, and undefined variable errors. The AI system provides feedback by highlighting the particular line of code that is responsible for the error.

Optimization Suggestions: The system identifies areas for improvement, which are code readability by using better naming conventions and comments, code optimization through better algorithms, and code maintainability by using modular code design.

6.5 Return Results to Frontend

The backend system formulates the answers after fulfilling its processing tasks since it has to provide the errors it has detected and their corresponding solutions for the particular problem it has identified. Its answers comprise the following:

- Error Type: The program detected a real problem which comprises two major problems, syntax errors, and missing import problems.
- Line Numbers: The exact line of code that contains issues.
- Suggested Fixes: These fixes are the suggested solutions to the current process.
- The system uses JSON to transmit feedback to the frontend, making it effective in the transmission of the data.

The continuous training is necessary in the AI backend system; this is because it has to learn something from each code submission made by the users. The machine learning models schedule their retraining process at fixed intervals in order to develop their capabilities with incoming code submissions and user feedback data.
The Three major elements on which the Enhanced Code Analysis And Feedback System is founded are: user authentication and code submission and AI-driven analysis. The system lets users submit their code right after they go through the login process via a text interface. In the back, the system makes use of the Gemini AI API for code processing in order to detect errors, which include syntax errors and logical problems, as well as missing imports. Optimization recommendations focus on readability improvement, improving performance, and easing code maintenance. It then presents this feedback to the user with concrete solutions they could implement. Following completion, the system keeps a track record of all reviews in its database, allowing users to go back and check their review progress.

7 RESULTS

The Enhanced Code Analysis and Feedback System provides real-time code feedback to users, therefore allowing for more efficient code review processes. The system utilizes the Gemini AI API to analyze submitted code, which detects both syntax errors and logical mistakes, and undefined variables and missing imports. This platform provides full feedback; it contains error descriptions and line numbers, optimization recommendations for code readability and performance, and maintainability. The platform allows developers to identify problems, therefore leading to better development of code. The system keeps track of all previous submissions along with their associated feedback, hence allowing users to keep track of their development. The AI system improves its performance through continued learning processes, developing more correct and relevant recommendation systems. It provides developers with tools that allow them to improve their efficiency at work while producing better software.

Figure1: Home interface

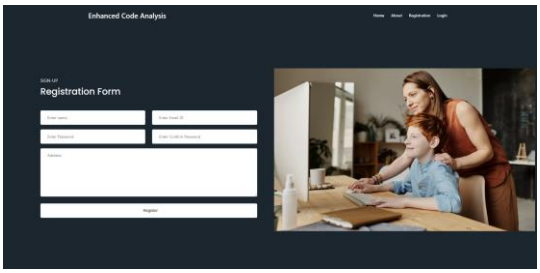


Figure 2: User sign-in and registration interface.

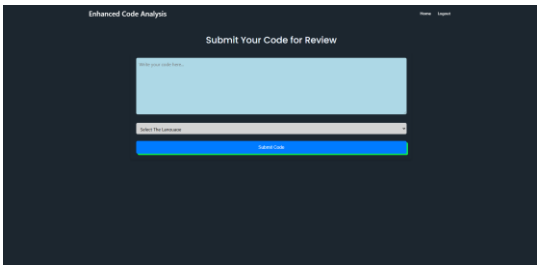


Figure 3: code review and feedback window.

8 DISCUSSION

From the results, it is clear that the benefits of adopting the proposed system involving artificial intelligence techniques along with the traditional static code analysis are significant. Manual code reviews are effective but usually have the limitations of time, personal biases, and code reviewer expertise.
The function of the AI-based component of reasoning is, therefore, important in enhancing explainability. Here, unlike other conventional tools that give confusing advice, the tool will help explain the problem and the solution to the problem, which is important in transferring knowledge and

enhancing the skills of programmers in developing better applications.

However, there are some limitations with this system. The performance of this AI model is conditional upon the quality of static analysis output as well as inputs given to it. In some complex and specific codes, some logical issues may need human intervention as well. There are some possibilities of latency issues due to API usage.

Despite the aforementioned limitations, the system reveals impressive prospects of application and usage in real-world development environments. Indeed, the system's scalability has impressive potential as a means of improving the existing code review process.

Furthermore, the outcome of this evaluation process emphasizes the value to be gained by implementing an automated process for code reviewing, as opposed to a manual method that relies not only on human skills but also on available time, as this might not be consistent for every individual.

The ability to generate explanatory feedback is an advantage, as it shifts the paradigm of code review from defect detection to a learning-focused mechanism. Code developers can learn from developing coding practices with improved logic and maintainability.

However, this is effective, and success depends on clear definitions of rules and interpretation of contexts. The complexity in terms of algorithmic logic may also call for human analysis and understanding. In addition, success with this system also depends on the input code.

The outcome of this study indicates that the Enhanced Code Analysis and Feedback Systems should be used in conjunction with human reviewers, as opposed to being used exclusively, as it strengthens the review approach as well as the quality of the development.

9 CONCLUSION AND FUTURE WORK

The Enhanced Code Analysis And Feedback System project explains how artificial intelligence technology benefits software development processes through application. The system, through its application, utilizes artificial intelligence technology, specifically Gemini AI API and machine learning models, to spot various programming mistakes made by developers and provide developers with this feedback. The platform allows developers to adapt improvements to their code that increase quality, readability, performance, and maintainability. Through this, the system generates new knowledge that allows it to provide more accurate recommendations over time as it operates. The tool assists developers in improving coding speed since it minimizes the amount of time they check through the coding process, thus improving coding standards within an organization. The improvement of the development process will incorporate IDE links into the project.

The Enhanced Code Analysis And Feedback System will develop new features through its integration with all main Integrated Development Environments, which provide support for VSCode and IntelliJ in order to enable developers to conduct live code assessments. The tool will gain more flexible functionality when its developers add additional programming language support. The system will achieve advanced error detection capabilities through deep learning methods that will let it find intricate problems. The implementation of collaborative code review features, including version tracking and team feedback, will facilitate teamwork and at the same time contribute to refining code quality. Implementation of AI-driven code suggestions for new frameworks and libraries will simplify the process of adopting new technologies in coding practices.

10 REFERENCES

Slasky, A. (2025, January 30). *20 best AI coding assistant tools [Updated Aug 2025]*. Qodo.ai Blog. https://www.qodo.ai/blog/best-ai-coding-assistant-tools

Vijayvergiya, M., et al. (2024, May 24). *AutoCommenter: AI-powered code review*. arXiv. https://arxiv.org/html/2405.13565v1

Weisz, J. D., & Vijay Kumar, S. (2025, April). *Examining the use and impact of an AI code assistant on developer productivity and experience in the enterprise*. CHI EA '25. https://dl.acm.org/doi/10.1145/3706599.3706670

Google Cloud. (2025, July 31). *Gemini code assist and GitHub AI code reviews*. Google Cloud Blog. https://cloud.google.com/blog/products/ai-machine-learning/gemini-code-assist-and-github-ai-code-reviews

Google Developers. (2025). *Gemini code assist overview*. https://developers.google.com/gemini-code-assist/docs/overview

AI code review assistant: A modern web-based solution for automated code analysis and developer productivity enhancement. (2025, September 2). ResearchGate. https://www.researchgate.net/publication/395116403_AI_Code_Review_Assistant_A_Modern_Web_Based_Solution_for_Automated_Code_Analysis_and_Developer_Productivity_Enhancement

AI-assisted assessment of coding practices in modern code review. (2024, May 24). arXiv. https://arxiv.org/html/2405.13565

AI-assisted code generation: Enhancing software development productivity with large language models. (2025, September 16). ResearchGate.

AI-powered code review assistant. (2024, December 24). IJISRT. https://www.ijisrt.com/assets/upload/files/IJISRT24DEC156.pdf

CodeRabbit. (2025). *AI code reviews | CodeRabbit | Try for free*. https://coderabbit.ai/